

Reconstruction de matrices Origine-Destination à partir de flux de voyageurs

Garance Malnoë
encadrée par Sylvain Faure et Bertrand Maury

Institut Mathématique d'Orsay

27 août 2024

Deux approches pour l'analyse des mouvements

Lagrangien

- Observation des entités individuellement.
- Exemple : Données GPS du téléphone.

Eulérien

- Observation des zones.
- Exemple : Comptage du nombre de passages à un arrêt.

Deux approches pour l'analyse des mouvements

Lagrangien

- Observation des entités individuellement.
- Exemple : Données GPS du téléphone.

Eulérien

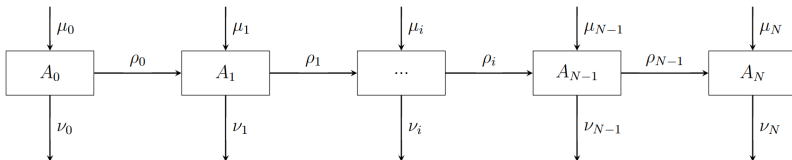
- Observation des zones.
- Exemple : Comptage du nombre de passages à un arrêt.

Passage de l'un à l'autre :

- Lagrangien à eulérien : simple grâce aux mesures.
- Eulérien à lagrangien : **plus compliqué.**

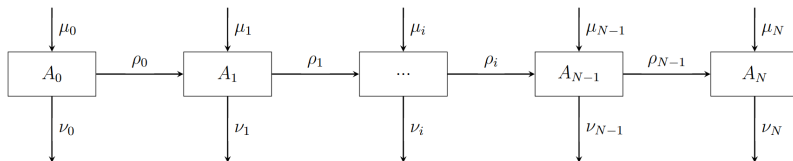
Cadre pour l'étude d'une ligne linéaire

- $N + 1$ arrêts de 0 à N .
- montées : $\mu_i \in \mathbb{N}$
 $\mu_N = 0$.
- descentes : $\nu_i \in \mathbb{N}$
 $\nu_0 = 0$.



Cadre pour l'étude d'une ligne linéaire

- $N + 1$ arrêts de 0 à N .
- montées : $\mu_i \in \mathbb{N}$
 $\mu_N = 0$.
- descentes : $\nu_i \in \mathbb{N}$
 $\nu_0 = 0$.



Problématique : retrouver la matrice Origine-Destination

$\gamma = (\gamma_{ij})_{i,j}$ où γ_{ij} = le nombre de personnes montant à l'arrêt i et descendant à l'arrêt j .

Algorithme naïf et algo de recherche exhaustive

Matrice correcte :

- $\forall i, j, \quad \gamma_{ij} \in \mathbb{N}.$
- $\forall i, \quad \sum_{j=0}^N \gamma_{ij} = \mu_i.$
- $\forall j, \quad \sum_{i=0}^N \gamma_{ij} = \nu_j.$

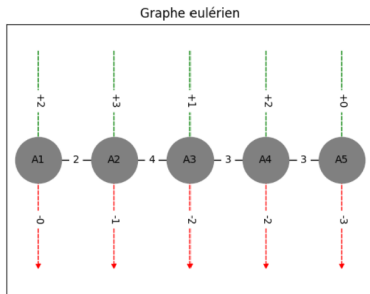


Figure: Ligne à 5 arrêts

Algorithme naïf et algo de recherche exhaustive

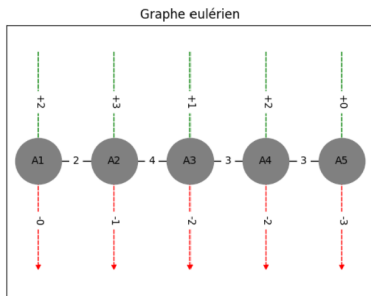


Figure: Ligne à 5 arrêts

Matrice correcte :

- $\forall i, j, \gamma_{ij} \in \mathbb{N}.$
- $\forall i, \sum_{j=0}^N \gamma_{ij} = \mu_i.$
- $\forall j, \sum_{i=0}^N \gamma_{ij} = \nu_j.$

Un des cinq résultats :

$$\gamma = \left(\begin{array}{ccccc|c} 0 & \mathbf{1} & 0 & \mathbf{1} & 0 & 2 \\ 0 & 0 & \mathbf{2} & 0 & \mathbf{1} & 3 \\ 0 & 0 & 0 & \mathbf{1} & 0 & 1 \\ 0 & 0 & 0 & 0 & \mathbf{2} & 2 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ - & - & - & - & - & + \\ \mathbf{0} & \mathbf{1} & \mathbf{2} & \mathbf{2} & \mathbf{3} & - \end{array} \right)$$

Résultats

	Nombre de solutions	Durée du calcul pour l'algorithme exhaustif (sec)
Ligne 5 arrêts, 8 voyageurs	5	0,02
Ligne 6 arrêts, 18 voyageurs	200	14,9

Figure: Nombre de solutions et temps de calcul pour deux exemples de lignes linéaires

Complexité de l'algorithme exhaustif dans le pire cas :

Nombre de coefficients

$$O(k N^2 / 2)$$

Nombre moyen de valeurs à tester
par coefficient

Sélectionner un scénario par l'entropie

Définition

On pose $K = \sum_{i,j} \gamma_{ij}$ le nombre total de voyageurs et $\bar{\gamma}_{ij} = \frac{\gamma_{ij}}{K}$.
L'entropie d'un plan Origine-Destination γ est défini comme :

$$S(\gamma) = \sum_{i,j} \bar{\gamma}_{ij} \log(\bar{\gamma}_{ij}).$$

Hypothèse : les voyageurs sont répartis uniformément sur les différents trajets $\longrightarrow$ on veut minimiser l'entropie.

Sélectionner un scénario par l'entropie

Définition

On pose $K = \sum_{i,j} \gamma_{ij}$ le nombre total de voyageurs et $\bar{\gamma}_{ij} = \frac{\gamma_{ij}}{K}$.
L'entropie d'un plan Origine-Destination γ est défini comme :

$$S(\gamma) = \sum_{i,j} \bar{\gamma}_{ij} \log(\bar{\gamma}_{ij}).$$

Hypothèse : les voyageurs sont répartis uniformément sur les différents trajets $\rightarrow$ on veut minimiser l'entropie.

Première approche : calcul sur les résultats de l'algorithme de recherche exhaustive $\rightarrow$ solution correcte, mais trop coûteux en temps de calcul.

Sélectionner un scénario par l'entropie

Définition

On pose $K = \sum_{i,j} \gamma_{ij}$ le nombre total de voyageurs et $\bar{\gamma}_{ij} = \frac{\gamma_{ij}}{K}$.
L'entropie d'un plan Origine-Destination γ est défini comme :

$$S(\gamma) = \sum_{i,j} \bar{\gamma}_{ij} \log(\bar{\gamma}_{ij}).$$

Hypothèse : les voyageurs sont répartis uniformément sur les différents trajets $\rightarrow$ on veut minimiser l'entropie.

Première approche : calcul sur les résultats de l'algorithme de recherche exhaustive $\rightarrow$ solution correcte, mais trop coûteux en temps de calcul.

Seconde approche : méthodes d'optimisation.

Minimisation sous contraintes

Remarque

Pour une ligne à N arrêts, il y a $d = \frac{N(N-1)}{2}$ données à trouver.

Exemple

$$\gamma = \begin{pmatrix} \gamma_{00} & \gamma_{01} & \gamma_{02} \\ \gamma_{10} & \gamma_{11} & \gamma_{12} \\ \gamma_{20} & \gamma_{21} & \gamma_{22} \end{pmatrix} = \begin{pmatrix} 0 & x_1 & x_2 \\ 0 & 0 & x_3 \\ 0 & 0 & 0 \end{pmatrix} \leftrightarrow x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

Définition

Soient $c_1, \dots, c_l \in C^1(\mathbb{R}^d, \mathbb{R})$ convexes appelées contraintes du problème. On pose :

$K = \{x \in \mathbb{R}^d \mid c_1(x) \leq 0, \dots, c_l(x) \leq 0\} \subseteq \mathbb{R}^d$, convexe fermé.

Théorie de l'optimisation $\longrightarrow$ **existence** et **unicité** d'une solution.

Contraintes pour la sélection d'une matrice Origine-Destination

Valeurs positives ou nulles :

- $c_i(x) = -x_i \leq 0$

Contraintes sur les montées :

- $c_{d+k}(x) = \sum_{j=1}^N \gamma_{kj} - \mu_k \leq 0$
- $c_{d+N+k}(x) = -(\sum_{j=1}^N \gamma_{kj} - \mu_k) \leq 0$

Contraintes sur les descentes :

- $c_{d+2N+k}(x) = \sum_{i=1}^N \gamma_{ik} - \nu_k \leq 0$
- $c_{d+3N+k}(x) = -(\sum_{i=1}^N \gamma_{ik} - \nu_k) \leq 0$

Méthode Trust-Constr

Méthode utilisée : `scipy.optimize.minimize` avec la méthode Trust-Constr (Trust Region Constrained Algorithms).

Arguments obligatoires:

- Bornes : $[0; +\infty[$
- Matrice des contraintes : (cf. rapport)
- Fonction à minimiser : $S(x) = \sum_{i=1}^n x_i \times \log(x_i)$

Arguments optionnels :

- Jacobienne : $\nabla S(x)_i = \log(x_i) + 1$
- Hessienne : $D^2 S(x)_{i,j} = \begin{cases} 0 & \text{si } i \neq j \\ \frac{1}{x_i} & \text{sinon.} \end{cases}$

Méthode de pénalisation

Nouvelle fonction avec contraintes intégrées :

$$\begin{aligned} S_{\epsilon}(x) = & \sum_{i=1}^N x_i \log(x_i) + \sum_{i=1}^N \frac{1}{\epsilon} \max(-x_i, 0)^2 \\ & + \sum_{i=1}^N \frac{1}{\epsilon} \left(\sum_{j \text{ sur la ligne } i} x_j - \mu_i \right)^2 \\ & + \sum_{j=1}^N \frac{1}{\epsilon} \left(\sum_{i \text{ sur la colonne } j} x_i - \nu_j \right)^2 \end{aligned}$$

Méthode utilisée : `scipy.optimize.minimize` avec la méthode Nedler-Mead.

Qualité de la prédiction sur des données réelles

Données : Matrice Origine-Destination pour les lignes a et b (15 arrêts), C4 (35 arrêts), C7 (19 arrêts)

Mesures:

- Moindres carrés : $Dist(A, B) = \sum_{i,j} (A_{i,j} - B_{i,j})^2$
- Entropie relative de μ par rapport à η :

$$S_{\eta}(\mu) = \sum_i \mu_i \log\left(\frac{\mu_i}{\eta_i}\right) = \sum_i \frac{\mu_i}{\eta_i} \log\left(\frac{\mu_i}{\eta_i}\right) \eta_i$$

- Comparaison avec croisement des marginales $\gamma_{i,j} = \mu_i \times \nu_j$

Qualité de la prédiction sur des données réelles

Données : Matrice Origine-Destination pour les lignes a et b (15 arrêts), C4 (35 arrêts), C7 (19 arrêts)

Mesures:

- Moindres carrés : $Dist(A, B) = \sum_{i,j} (A_{i,j} - B_{i,j})^2$

- Entropie relative de μ par rapport à η :

$$S_{\eta}(\mu) = \sum_i \mu_i \log\left(\frac{\mu_i}{\eta_i}\right) = \sum_i \frac{\mu_i}{\eta_i} \log\left(\frac{\mu_i}{\eta_i}\right) \eta_i$$

- Comparaison avec croisement des marginales $\gamma_{i,j} = \mu_i \times \nu_j$

Résultats :

- Forte augmentation du temps de calcul pour Trust-Constr (C7: 1min, C4: 10min) contre <2s pour Penalisation-NM
- Moindres carrés : inconclusif.
- Entropie relative : Pénalisation NM → Amélioration de 40% par rapport au croisement des marginales (mais triche).

Conclusion

- Résultats mitigés pour la recherche de l'ensemble des solutions et pour la sélection d'une solution.
- Plusieurs pistes d'amélioration : changer de langage, changer d'hypothèse sur l'entropie, autres méthodes Scipy, ...
- Adaptation d'un problème similaire avec des méthodes de probas/stats (cf. rapport).